

Chat

Implementar um chat em tempo real usando Java, Docker e Kafka envolve várias etapas. Vamos criar um sistema simples de chat com os seguintes componentes:

1. **Producer Service:** Um serviço que envia mensagens para um tópico Kafka.
2. **Consumer Service:** Um serviço que lê mensagens de um tópico Kafka e as transmite para os clientes.
3. **WebSocket Server:** Um servidor que permite a comunicação em tempo real entre o cliente e o servidor.
4. **Frontend:** Uma aplicação web simples para o chat.

Passo a Passo

1. Configurar o Ambiente de Desenvolvimento

- Instale o JDK, Maven, Docker e Kafka.
- Para facilitar, vamos usar o Docker Compose para configurar o Kafka.

Sugestão de organização de arquivos e diretórios:

```
.
├── consumer-service
│   ├── Dockerfile
│   ├── pom.xml
│   └── src
│       ├── main
│       │   ├── java
│       │   │   ├── br
│       │   │   │   ├── com
│       │   │   │   │   ├── meslin
│       │   │   │   │   │   ├── ChatConsumer.java
│       │   │   │   │   │   └── WebSocketServer.java
│       └── target
│           ├── classes
│           │   ├── br
│           │   │   ├── com
│           │   │   │   ├── meslin
│           │   │   │   │   ├── ChatConsumer.class
│           │   │   │   │   └── WebSocketServer.class
│           └── consumer-service-jar-with-dependencies.jar
├── docker-compose.yaml
├── frontend
│   ├── chat_listener.py
│   ├── chat_sender.py
│   ├── index.html
│   └── requirements.txt
└── producer-service
    ├── Dockerfile
    ├── pom.xml
    └── src
        ├── main
        │   ├── java
        │   │   ├── br
        │   │   │   ├── com
        │   │   │   │   ├── meslin
        │   │   │   │   │   ├── ChatProducer.java
        │   │   │   │   │   └── WebSocketServer.java
```

```

├── target
│   ├── archive-tmp
│   ├── classes
│   │   ├── br
│   │   │   ├── com
│   │   │   │   ├── meslin
│   │   │   │   │   ├── ChatProducer.class
│   │   │   │   │   └── WebSocketServer.class
│   └── producer-service-jar-with-dependencies.jar
├── start.sh
└── stop.sh

```

2. Criar o Producer Service

a. Dependências Maven

Adicione as dependências do Kafka e WebSocket ao seu pom.xml:

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>br.com.meslin</groupId>    <!-- Grupo do projeto -->
  <artifactId>producer-service</artifactId>    <!-- Nome do artefato -->
  <version>1.0-SNAPSHOT</version>    <!-- Versão do projeto -->

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
  </properties>

  <build>
    <finalName>producer-service</finalName>

    <plugins>
      <plugin> <!-- Configuração do plugin de compilação do Maven -->
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.8.1</version>
        <configuration>
          <source>11</source>
          <target>11</target>
          <release>11</release>
        </configuration>
      </plugin>
      <plugin> <!-- Configuração do plugin de empacotamento do Maven -->
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.2.0</version>
        <executions>
          <execution>
            <id>default-jar</id>
            <phase>none</phase> <!-- Isso desativa a criação do JAR padrão -->
          </execution>
        </executions>
      </plugin>
      <plugin> <!-- Configuração do plugin de montagem do Maven para criar um JAR com
dependências -->

```

```

    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-assembly-plugin</artifactId>
    <version>3.3.0</version>
    <configuration>
        <archive>
            <manifest>
                <mainClass>br.com.meslin.ChatProducer</mainClass> <!-- Substitua com o nome
da sua classe principal -->
            </manifest>
        </archive>
        <descriptorRefs>
            <descriptorRef>jar-with-dependencies</descriptorRef>
        </descriptorRefs>
    </configuration>
    <executions>
        <execution>
            <id>make-assembly</id>
            <phase>package</phase>
            <goals>
                <goal>single</goal>
            </goals>
        </execution>
    </executions>
</plugin>
</plugins>
</build>

<dependencies>
    <!-- Apache Kafka Client -->
    <dependency>
        <groupId>org.apache.kafka</groupId>
        <artifactId>kafka-clients</artifactId>
        <version>3.8.0</version>
    </dependency>

    <dependency>
        <groupId>org.glassfish.tyrus.bundles</groupId>
        <artifactId>tyrus-standalone-client</artifactId>
        <version>1.13.1</version>
    </dependency>

    <!-- https://mvnrepository.com/artifact/org.glassfish.tyrus/tyrus-container-grizzly -
->
    <dependency>
        <groupId>org.glassfish.tyrus</groupId>
        <artifactId>tyrus-container-grizzly</artifactId>
        <version>1.2.1</version>
        <scope>test</scope>
    </dependency>

    <!-- From: https://stackoverflow.com/questions/38284762/classnotfoundexception-when-
start-tyrus-standalone-server -->
    <dependency>
        <groupId>org.glassfish.tyrus</groupId>
        <artifactId>tyrus-container-grizzly-server</artifactId>
        <version>1.13</version>
    </dependency>

    <!-- SLF4J API -->
    <!-- https://mvnrepository.com/artifact/org.slf4j/slf4j-api -->
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-api</artifactId>

```

```

        <version>2.0.13</version>
    </dependency>

    <!-- SLF4J Simple Binding (or another binding implementation) -->
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-simple</artifactId>
        <version>2.0.13</version>
    </dependency>
</dependencies>
</project>

```

b. Implementar o Producer

Crie a classe ChatProducer:

```

/**
 * Classe que representa o produtor do chat.
 *
 * @author Alexandre Meslin
 * @version 1.0
 * @since 2024-06-10
 */
package br.com.meslin;

import org.apache.kafka.clients.producer.KafkaProducer;
import org.apache.kafka.clients.producer.ProducerRecord;
import org.apache.kafka.clients.producer.ProducerConfig;
import org.apache.kafka.common.serialization.StringSerializer;

import java.util.Properties;
import java.util.Date;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class ChatProducer {
    private final KafkaProducer<String, String> producer;    /// Instância do produtor
    Kafka para enviar mensagens.
    private final String topic;    /// Nome do tópico Kafka para o qual as mensagens
    serão enviadas.

    private static final Logger logger = LoggerFactory.getLogger(ChatProducer.class);
    /// Instância do logger para registrar informações e erros.

    /**
     * Constructor for ChatProducer.
     *
     * @param topic The Kafka topic to which messages will be sent.
     */
    public ChatProducer(String topic) {
        Properties props = new Properties();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "kafka:9092");
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
StringSerializer.class.getName());

        producer = new KafkaProducer<>(props);
        this.topic = topic;
    }
}

```

```

/**
 * Sends a message to the Kafka topic.
 *
 * @param message The message to be sent.
 */
public void sendMessage(String message) {
    logger.info("[ChatProducer.sendMessage] " + message);
    message = new Date() + " ==> " + message;
    logger.info("[ChatProducer.sendMessage] " + message);
    producer.send(new ProducerRecord<>(topic, message));
}

/**
 * Closes the Kafka producer to release resources.
 */
public void close() {
    producer.close();
}

/**
 * Main method to start the ChatProducer and WebSocket server.
 *
 * @param args Command line arguments (not used).
 */
public static void main(String[] args) {
    logger.info("[ChatProducer.main] Starting Chat Producer.");

    // Initialize ChatProducer with the topic "chat-messages"
    ChatProducer chatProducer = new ChatProducer("chat-messages");

    // Start the WebSocket server and pass the ChatProducer instance to it
    WebSocketServer.startServer(chatProducer);

    // Add a shutdown hook to gracefully close the producer and stop the WebSocket
server
    Runtime.getRuntime().addShutdownHook(new Thread(() -> {
        logger.info("[ChatProducer.main] Shutting down...");
        WebSocketServer.stopServer();
        chatProducer.close();
        logger.info("[ChatProducer.main] Shutdown complete.");
    }));

    // Keep the application running
    try {
        // Use a synchronized block to wait indefinitely
        synchronized (ChatProducer.class) {
            ChatProducer.class.wait();
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
}

```

c. Implementar o WebSocket Server

Crie a classe `WebSocketServer`:

```

/**
 * WebSocketServer.java
 * This class implements a WebSocket server that handles incoming messages and broadcasts
them to all connected clients.

```

```

* It uses the ChatProducer class to send messages to a Kafka topic.
* The server listens for WebSocket connections on the "/ws" endpoint.
* The server address is "localhost" and the port is 8080, with the context path "/chat".
* Full server address: ws://localhost:8080/chat/ws
*
* @author Meslin
* @version 1.0
* @since 2024-06-10
*/
package br.com.meslin;

import javax.websocket.OnClose;
import javax.websocket.OnMessage;
import javax.websocket.OnOpen;
import javax.websocket.Session;
import javax.websocket.server.ServerEndpoint;
import java.io.IOException;
import java.util.Set;
import java.util.concurrent.CopyOnWriteArraySet;
import org.glassfish.tyrus.server.Server;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import java.util.Date;

@ServerEndpoint("/ws") // Acrescenta /ws ao endereço do servidor WebSocket
public class WebSocketServer {
    private static ChatProducer chatProducer;    /// Instância do ChatProducer para enviar
    mensagens para o Kafka.
    private Session session;                    /// Instância da sessão WebSocket associada a
    esta conexão.
    private static final Set<WebSocketServer> connections = new CopyOnWriteArraySet<>();
    /// Conjunto de conexões WebSocket ativas.
    private static Server server;                /// Instância do servidor WebSocket.
    private static final Logger logger = LoggerFactory.getLogger(ChatProducer.class);
    /// Instância do logger para registrar informações e erros.

    /**
     * Starts the WebSocket server and initializes it with the provided ChatProducer
     instance.
     *
     * @param producer The ChatProducer instance to be used for sending messages to
     Kafka.
     */
    public static void startServer(ChatProducer producer) {
        logger.info("[WebSocketServer.startServer] Starting WebSocket Server.");
        chatProducer = producer;
        server = new Server("localhost", 8080, "/chat", null, WebSocketServer.class);

        try {
            server.start();
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }

    /**
     * Stops the WebSocket server and releases any associated resources.
     * This method should be called when the application is shutting down.
     * It ensures that the WebSocket server is properly stopped and any active
     connections are closed.
     */
    public static void stopServer() {

```

```

        logger.info("[WebSocketServer.stopServer] Stopping WebSocket Server.");
        server.stop();
    }

    /**
     * Handles the event when a new WebSocket connection is established.
     * This method is called when a client connects to the WebSocket server.
     * It adds the new connection to the set of active connections and logs the event.
     *
     * @param session    The WebSocket session associated with the new connection.
     */
    @OnOpen
    public void onOpen(Session session) {
        logger.info("[WebSocketServer.onOpen] New connection established.");
        this.session = session;
        connections.add(this);
    }

    /**
     * Handles incoming messages from WebSocket clients.
     * This method is called when a client sends a message to the WebSocket server.
     * It logs the received message, sends it to the Kafka topic using the ChatProducer,
     * and broadcasts the message to all connected WebSocket clients.
     *
     * @param message    The message received from the client.
     */
    @OnMessage
    public void onMessage(String message) {
        logger.info("[WebSocketServer.onMessage] Received message: " + message);
        // Send the message to Kafka
        chatProducer.sendMessage(message);
        // Broadcast the message to all connected WebSocket clients
        broadcast(message);
    }

    /**
     * Handles the event when a WebSocket connection is closed.
     * This method is called when a client disconnects from the WebSocket server.
     * It removes the connection from the set of active connections and logs the event.
     *
     * @param session    The WebSocket session associated with the closed connection.
     */
    @OnClose
    public void onClose(Session session) {
        logger.info("[WebSocketServer.onClose] Connection closed.");
        connections.remove(this);
    }

    /**
     * Broadcasts a message to all connected WebSocket clients.
     * This method iterates through all active WebSocket connections and sends the
    provided message
     * to each client. If an error occurs while sending the message to a client,
     * that client is removed from the set of active connections.
     *
     * @param message    The message to broadcast.
     */
    public static void broadcast(String message) {
        logger.info("[WebSocketServer.broadcast] Broadcasting message: " + message);
        for (WebSocketServer client : connections) {
            try {
                synchronized (client) {

```

```
        client.session.getBasicRemote().sendText(new Date() + " --> " +  
message);  
    }  
} catch (IOException e) {  
    connections.remove(client);  
    try {  
        client.session.close();  
    } catch (IOException ex) {  
        // Ignore  
    }  
}  
}  
}
```

3. Criar o Consumer Service

a. Dependências Maven

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>br.com.meslin</groupId>
  <artifactId>consumer-service</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>

  <build>
    <finalName>consumer-service</finalName>
    <sourceDirectory>src</sourceDirectory>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <version>3.8.1</version>
        <configuration>
          <source>11</source>
          <target>11</target>
          <release>11</release>
        </configuration>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.2.0</version>
        <executions>
          <execution>
            <id>default-jar</id>
            <phase>none</phase> <!-- Isso desativa a criação do JAR padrão -->
          </execution>
        </executions>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
```

```

        <artifactId>maven-assembly-plugin</artifactId>
        <version>3.3.0</version>
        <configuration>
            <archive>
                <manifest>
                    <mainClass>br.com.meslin.ChatConsumer</mainClass> <!-- Substitua
com o nome da sua classe principal -->
                </manifest>
            </archive>
            <descriptorRefs>
                <descriptorRef>jar-with-dependencies</descriptorRef>
            </descriptorRefs>
        </configuration>
        <executions>
            <execution>
                <id>make-assembly</id>
                <phase>package</phase>
                <goals>
                    <goal>single</goal>
                </goals>
            </execution>
        </executions>
    </plugin>
</plugins>
</build>

<dependencies>
    <dependency>
        <groupId>org.apache.kafka</groupId>
        <artifactId>kafka-clients</artifactId>
        <version>3.8.0</version>
    </dependency>
    <dependency>
        <groupId>javax.websocket</groupId>
        <artifactId>javax.websocket-api</artifactId>
        <version>1.1</version>
    </dependency>
    <dependency>
        <groupId>org.glassfish.tyrus</groupId>
        <artifactId>tyrus-server</artifactId>
        <version>1.13.1</version>
    </dependency>
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-api</artifactId>
        <version>1.7.24</version>
    </dependency>
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-simple</artifactId>
        <version>1.7.24</version>
    </dependency>
    <dependency>
        <groupId>org.glassfish.tyrus</groupId>
        <artifactId>tyrus-container-grizzly-server</artifactId>
        <version>1.13</version>
    </dependency>
</dependencies>
</project>

```

b. Implementar o Consumer

Crie a classe ChatConsumer:

```
/**
 * ChatConsumer.java
 * This class implements a Kafka consumer that listens for messages on the "chat-
 messages" topic.
 * It receives messages from Kafka and broadcasts them to all connected WebSocket
 clients.
 *
 * @author Meslin
 * @version 1.0
 * @since 2024-06-10
 */
package br.com.meslin;

import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.clients.consumer.ConsumerRecords;
import org.apache.kafka.clients.consumer.KafkaConsumer;

import java.time.Duration;
import java.util.Collections;
import java.util.Properties;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

import br.com.meslin.WebSocketServer;

public class ChatConsumer {
    private static final String TOPIC = "chat-messages";          /// Nome do tópico
 Kafka do chat.
    private static final String BOOTSTRAP_SERVERS = "kafka:9092"; /// Endereços dos
 servidores Kafka.
    private static final String GROUP_ID = "chat-consumer-group"; /// ID do grupo de
 consumidores Kafka.

    private static final Logger logger = LoggerFactory.getLogger(ChatConsumer.class);
    /// Instância do logger para registrar informações e erros.

    /**
     * Main method to start the ChatConsumer and WebSocket server.
     * It initializes the Kafka consumer, subscribes to the "chat-messages" topic,
     * and continuously polls for new messages.
     * Received messages are broadcasted to all connected WebSocket clients.
     *
     * @param args Command line arguments (not used).
     */
    public static void main(String[] args) {
        logger.info("Starting Chat Consumer.");
        Properties props = new Properties();
        props.put("bootstrap.servers", BOOTSTRAP_SERVERS);
        props.put("group.id", GROUP_ID);
        props.put("key.deserializer",
 "org.apache.kafka.common.serialization.StringDeserializer");
        props.put("value.deserializer",
 "org.apache.kafka.common.serialization.StringDeserializer");

        KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);
        consumer.subscribe(Collections.singletonList(TOPIC));

        WebSocketServer.startServer();
    }
}
```

```

        try {
            while (true) {
                ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(100));
                for (ConsumerRecord<String, String> record : records) {
                    logger.info("Received message: " + record.value());
                    WebSocketServer.broadcast(record.value());
                }
            }
        } finally {
            consumer.close();
        }
    }
}

```

c. Implementar o Webservice

Crie a classe WebSocketServer:

```

/**
 * WebSocketServer.java
 * This class implements a WebSocket server that handles real-time communication between
clients and the application.
 * It manages WebSocket connections, processes incoming messages, and broadcasts messages
to all connected clients.
 *
 * @author Meslin
 * @version 1.0
 * @since 2024-06-10
 */
package br.com.meslin;

import javax.websocket.OnClose;
import javax.websocket.OnMessage;
import javax.websocket.OnOpen;
import javax.websocket.Session;
import javax.websocket.server.ServerEndpoint;
import java.io.IOException;
import java.util.Set;
import java.util.concurrent.CopyOnWriteArraySet;
import org.glassfish.tyrus.server.Server;

import java.util.Date;

@ServerEndpoint(value = "/ws")
public class WebSocketServer {

    private Session session;          /// Instância da sessão WebSocket associada a
esta conexão.
    private static final Set<WebSocketServer> connections = new CopyOnWriteArraySet<>();
/// Conjunto de conexões WebSocket ativas.
    private static Server server;      /// Instância do servidor WebSocket.

    /**
     * Starts the WebSocket server.
     * This method initializes the WebSocket server and starts it on the specified host
and port.
     */
    public static void startServer() {
        server = new Server("localhost", 8080, "/chat", null, WebSocketServer.class);

        try {

```

```

        server.start();
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}

/**
 * Stops the WebSocket server.
 * This method stops the WebSocket server and releases any associated resources.
 */
public static void stopServer() {
    server.stop();
}

/**
 * Handles the event when a new WebSocket connection is established.
 * This method is called when a client connects to the WebSocket server.
 * It adds the new connection to the set of active connections.
 *
 * @param session    The WebSocket session associated with the new connection.
 */
@OnOpen
public void onOpen(Session session) {
    this.session = session;
    connections.add(this);
}

/**
 * Handles incoming messages from WebSocket clients.
 * This method is called when a client sends a message to the WebSocket server.
 * It broadcasts the received message to all connected WebSocket clients.
 *
 * @param message    The message received from the client.
 */
@OnMessage
public void onMessage(String message) {

    broadcast(message);
}

/**
 * Handles the event when a WebSocket connection is closed.
 * This method is called when a client disconnects from the WebSocket server.
 * It removes the connection from the set of active connections.
 *
 * @param session    The WebSocket session associated with the closed connection.
 */
@OnClose
public void onClose(Session session) {
    connections.remove(this);
}

/**
 * Broadcasts a message to all connected WebSocket clients.
 * This method iterates through all active WebSocket connections and sends the
provided message
 * to each client. If an error occurs while sending the message to a client,
 * that client is removed from the set of active connections.
 *
 * @param message    The message to broadcast.
 */
public static void broadcast(String message) {
    for (WebSocketServer client : connections) {

```

```

        try {
            synchronized (client) {
                client.session.getBasicRemote().sendText(message);
            }
        } catch (IOException e) {
            connections.remove(client);
            try {
                client.session.close();
            } catch (IOException ex) {
                // Ignore
            }
        }
    }
}
}
}
}

```

4. Criar Dockerfiles para os Serviços

Dockerfile para o Producer Service

Crie um arquivo `Dockerfile` no diretório do Producer Service:

```

FROM eclipse-temurin:11-jre

WORKDIR /app

COPY target/producer-service-jar-with-dependencies.jar producer-service.jar

CMD ["java", "-jar", "producer-service.jar"]

```

Crie um arquivo `Dockerfile` no diretório do Consumer Service:

```

FROM eclipse-temurin:11-jre

WORKDIR /app

COPY target/consumer-service-jar-with-dependencies.jar consumer-service.jar

CMD ["java", "-jar", "consumer-service.jar"]

```

5. Compilar as aplicações

No diretório de cada serviço, execute:

```
mvn package
```

6. Criar arquivo YAML para o Docker Compose

Crie um arquivo `docker-compose.yaml` na raiz do projeto:

```

services:
  kafka:
    # Nome do serviço Kafka
    image: apache/kafka:latest # Imagem oficial do Kafka
    #container_name: kafka     # Nome do container
    ports:
      # Mapeamento de portas
      - "9092:9092"            # Porta para comunicação com clientes Kafka

```

```
environment:    # Variáveis de ambiente para configuração do Kafka
  # FROM: https://hub.docker.com/r/apache/kafka
  # Kafka em modo KRaft: broker + controller
  KAFKA_NODE_ID: 1          # Identificador único do nó Kafka
  KAFKA_PROCESS_ROLES: broker,controller # Define os papéis do nó Kafka (broker e
controller)

  # Controller interno
  KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093 # KAFKA_CONTROLLER_QUORUM_VOTERS is a
comma-separated list of the controllers

  # Listeners
  KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
  KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092

  # Protocolos dos listeners
  KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT

  # Listener usado pelos brokers para comunicação interna
  KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT

  # Listener usado pelo controller
  KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER

  # Configurações para um único broker
  KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
  KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
  KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1

  # Criar tópicos automaticamente
  KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"

volumes:    # Montagem de volume para persistência de dados do Kafka
  - kafka-data:/var/lib/kafka/data

deploy:
  resources:
    limits:
      memory: 1G
    reservations:
      memory: 512M

networks:
  - int-network

producer-service:
  build: ./producer-service
  ports:
    - "8080:8080"

depends_on:
  - kafka

deploy:
  resources:
    limits:
      memory: 512M
    reservations:
      memory: 256M

networks:
  - int-network
```

```

    - ext-network

consumer-service:
  build: ./consumer-service
  ports:
    - "8081:8080"

  depends_on:
    - kafka

  deploy:
    resources:
      limits:
        memory: 512M
      reservations:
        memory: 256M

  networks:
    - ext-network
    - int-network

volumes:
  kafka-data:

networks:
  ext-network:
    driver: bridge

  int-network:
    driver: bridge
    internal: true

```

7. Configurar Kafka com Docker Compose

Inicie a aplicação:

```
docker-compose up -d
```

Verifique se as imagens estão no ar:

```
$ sudo docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED
STATUS			
NAMES			
a657b88c56a5	wurstmeister/kafka:latest	"start-kafka.sh"	32 minutes ago
Exited (1) 32 minutes ago			
chat_kafka_1			

8. Criar o Frontend

Se você não estiver usando o Codespace para fazer o deploy, crie um arquivo HTML simples para o frontend:

```

<!DOCTYPE html>
<html>
<head>
  <title>Chat</title>
  <script>
    let wsProducer;

```

```

    let wsConsumer;

    /**
     * Initializes WebSocket connections to the producer and consumer servers.
     * This function is called when the DOM content is fully loaded.
     * It dynamically constructs the WebSocket URLs based on the current host and
connects to both servers.
     * Incoming messages from each server are displayed in their respective chat
boxes.
     * @param {void} - No parameters.
     */
    document.addEventListener('DOMContentLoaded', function() {
        // Pega o nome do host da página atual (ex: localhost, meuchat.com,
192.168.1.100)
        const serverAddress = window.location.hostname;
        const serverAddress8080 = serverAddress.replace("-8000", "-8080");
        const serverAddress8081 = serverAddress.replace("-8000", "-8081");

        console.log(`Tentando conexão com ${serverAddress8080} na porta 8080`);
        console.log(`Tentando conexão com ${serverAddress8081} na porta 8081`);

        // Constrói o URL do WebSocket dinamicamente
        wsProducer = new WebSocket(`ws://${serverAddress8080}/chat/ws`);
        wsConsumer = new WebSocket(`ws://${serverAddress8081}/chat/ws`);

        // Define o que acontece quando uma mensagem é recebida do servidor produtor
        wsProducer.onmessage = function(event) {
            let chatbox = document.getElementById("chatboxproducer");
            chatbox.innerHTML += event.data + "<br/>";
        };

        wsConsumer.onmessage = function(event) {
            let chatbox = document.getElementById("chatboxconsumer");
            chatbox.innerHTML += event.data + "<br/>";
        };
    });

    function sendMessage() {
        let message = document.getElementById("message").value;
        wsProducer.send(message);
    }
</script>
</head>
<body>
    <h1>Chat</h1>
    <div id="chatboxproducer"></div>
    <input type="text" id="message" placeholder="Type a message..." />
    <button onclick="sendMessage()">Send</button>
    <div id="chatboxconsumer"></div>
</body>
</html>

```

Coloque este arquivo em um servidor web (por exemplo, usando Python para simplicidade):

```
python3 -m http.server 8000
```

Conclusão

Este exemplo mostra como criar um chat em tempo real usando Java, Docker e Kafka. Ele inclui a configuração do Kafka usando Docker Compose, a implementação de serviços Producer e Consumer em Java, e a

configuração de WebSocket para comunicação em tempo real. Este exemplo pode ser expandido e melhorado para incluir mais funcionalidades e otimizações.